

Chapter 22. Vectorized String operations in Pandas

Pandas includes features to handle vectorized string operations. This is achieved via the `.str` attribute of a `Series` or `Index` objects.

For example, the following will apply capitalization to all strings in a series, and properly handle *missing data*:

In [1]:

```
import pandas as pd

data = ['peter', 'Paul', None, 'MARY', 'gUIDO'] # Note the missing name in the third position.

names = pd.Series(data)
names.str.capitalize()
```

Out[1]:

```
0    Peter
1     Paul
2     None
3     Mary
4    Guido
dtype: object
```

Pandas String methods

Many of Pandas' string methods mirror the built-in Python string functions. Some of the most common are: `len`, `lower`, `upper`, `islower`, `isupper`, `find`, `endswith`, `startswith`, `isnumeric`, `isalnum`, `isdecimal`, `zfill`, `isalpha`, `split`, `strip`, `rstrip`, `lstrip`, `isdigit`, `capitalize`, `isspace`.

Various of these methods return values. These could be string values, e.g.:

In [4]:

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',  
                  'Eric Idle', 'Terry Jones', 'Michael Palin'])  
  
monte.str.lower()  
monte.str.len()  
monte.str.find('ee')
```

Out[4]:

```
0    -1  
1     7  
2    -1  
3    -1  
4    -1  
5    -1  
dtype: int64
```

Other methods return numbers:

In [5]:

```
monte.str.len()
```

Out[5]:

```
0    14
1    11
2    13
3     9
4    11
5    13
dtype: int64
```

And others return Booleans:

In [6]:

```
monte.str.startswith('T')
```

Out[6]:

```
0    False
1    False
2     True
3    False
4     True
5    False
dtype: bool
```

Still others return lists or other compound values:

In [7]:

```
monte.str.split()
```

Out[7]:

```
0    [Graham, Chapman]
1    [John, Cleese]
2    [Terry, Gilliam]
3    [Eric, Idle]
4    [Terry, Jones]
5    [Michael, Palin]
dtype: object
```

Regular expressions

In addition to standard string methods, Pandas also implements regular expression (re) matching to string objects, for example:

- `match` - Calls `re.match`, searching for a pattern in the string.
- `replace` - Replaces occurrences of pattern with some other string.
- `count` - Counts occurrences of an input pattern in a string.

For instance, the following extract the first name from each element in the Series:

In [24]:

```
monte.str.extract('([A-Za-z]+)', expand=False)
monte.str.replace('([A-Za-z])', '-', regex=True)
```

Out[24]:

```
0      -----
1      ----
2      -----
3      ----
4      -----
5      -----
dtype: object
```

Vectorized string slicing

It is possible to perform vectorized slicing using the `.str` attribute in a Pandas `Series`. The following returns a `Series` with only the first three characters of each name in the original `Series`:

In []:

```
monte.str[0:3]
```

Chapter 23. Time series

Native Python dates and times: `datetime` and `dateutil`

The `datetime` module allows the manipulation of dates and times in python:

In [25]:

```
from datetime import datetime  
  
datetime(year=2021, month=7, day=4)
```

Out[25]:

```
datetime.datetime(2021, 7, 4, 0, 0)
```

The `dateutil` module has interfaces to automatically convert strings in certain format into `datetime` objects.

In []:

```
from dateutil import parser  
  
date = parser.parse("4th of July, 2021")  
date
```

NumPy's `datetime64`

NumPy provides a data type for handling datetimes with 64-bits of precision:

In [26]:

```
import numpy as np  
  
date = np.array('2021-07-04', dtype=np.datetime64)  
date
```

Out[26]:

```
array('2021-07-04', dtype='datetime64[D]')
```

Once a NumPy datetime object is created, we can apply vectorized operations to it:

In [27]:

```
date + np.arange(12)
```

Out[27]:

```
array(['2021-07-04', '2021-07-05', '2021-07-06', '2021-07-07',  
      '2021-07-08', '2021-07-09', '2021-07-10', '2021-07-11',  
      '2021-07-12', '2021-07-13', '2021-07-14', '2021-07-15'],  
      dtype='datetime64[D]')
```

Dates and times in Pandas

Pandas offers an interface for handling dates and times similar to `np.datetime64`:

In [40]:

```
import pandas as pd  
  
date = pd.to_datetime("4th of July, 2021, 1:00 PM")  
date
```

Out[40]:

```
Timestamp('2021-07-04 13:00:00')
```

We can use `.strftime()` to print a `datetime` object as a string. The following gets the day of the week (`'%A'`) from a `datetime`:

In [47]:

```
date.strftime('%Y-%m-%d %H:%M:%S')
```

Out[47]:

```
'2021-07-04 13:00:00'
```

Pandas also allows for vectorized date manipulation:

In [48]:

```
date + pd.to_timedelta(np.arange(12), 'D')
```

Out[48]:

```
DatetimeIndex(['2021-07-04 13:00:00', '2021-07-05 13:00:00',  
               '2021-07-06 13:00:00', '2021-07-07 13:00:00',  
               '2021-07-08 13:00:00', '2021-07-09 13:00:00',  
               '2021-07-10 13:00:00', '2021-07-11 13:00:00',  
               '2021-07-12 13:00:00', '2021-07-13 13:00:00',  
               '2021-07-14 13:00:00', '2021-07-15 13:00:00'],  
              dtype='datetime64[ns]', freq=None)
```

Pandas Time Series: Indexing by time

Let us construct a `Series` object that has time-indexed data:

In [49]:

```
index = pd.DatetimeIndex(['2020-07-04', '2020-08-04',  
                           '2021-07-04', '2021-08-04'])  
data = pd.Series([0, 1, 2, 3], index=index)  
data
```

Out[49]:

```
2020-07-04    0  
2020-08-04    1  
2021-07-04    2  
2021-08-04    3  
dtype: int64
```

Now that the dates are indices, we can use `Series` indexing patterns to filter the data, coercing strings into dates:

In [50]:

```
data['2020-07-04':'2021-07-04']
```

Out[50]:

```
2020-07-04    0  
2020-08-04    1  
2021-07-04    2  
dtype: int64
```

There are special indexing operations, such as passing a year. Doing this will return a slice of the data from that year:

In [51]:

```
data['2021']
```

Out[51]:

```
2021-07-04    2
2021-08-04    3
dtype: int64
```

Pandas time series data structures

- For *timestamps*, Pandas provides the `Timestamp` type.
- For *time periods*, Pandas provides the `Period` type.
- For *time deltas* or *durations*, Pandas provides the `Timedelta` type.

Creating `Timestamp` objects with `.to_datetime`:

In [54]:

```
dates = pd.to_datetime([datetime(2021, 7, 3), '4th of July, 2021',
                                '2021-Jul-6', '07-07-2021', '20210708'])
dates
```

Out[54]:

```
DatetimeIndex(['2021-07-03', '2021-07-04', '2021-07-06', '2021-07-07',  
              '2021-07-08'],  
              dtype='datetime64[ns]', freq=None)
```

You can also provide a format to be interpreted by Pandas `to_datetime`. The `format` argument expects a string describing how the date string is constructed (this is also called `strftime`). Using placeholders as:

- `'%Y'` : year with 4 digits.
- `'%d'` : day of month with 2 digits (1-31).
- `'%m'` : numerical representation of the month with 2 digits (1-12).
- `'%H'` : hours in 24h format.
- `'%M'` : minutes with 2 digits.
- `'%S'` : seconds with 2 digits.

Example:

In [52]:

```
dates_fmt = pd.to_datetime(['2025-02-03', '2025-11-01', '2025-01-11'], format='%Y-%d-%m')  
dates_fmt
```

Out[52]:

```
DatetimeIndex(['2025-03-02', '2025-01-11', '2025-11-01'], dtype='datetime64[n  
s]', freq=None)
```

Converting Timestamp to Period. 'D' indicates that the periods correspond to one day.

In [55]:

```
dates.to_period('D')
```

Out[55]:

```
PeriodIndex(['2021-07-03', '2021-07-04', '2021-07-06', '2021-07-07',  
            '2021-07-08'],  
            dtype='period[D]')
```

a `TimedeltaIndex` is created when a date is subtracted from another:

In [56]:

```
dates - dates[0]
```

Out[56]:

```
TimedeltaIndex(['0 days', '1 days', '3 days', '4 days', '5 days'], dtype='timedelta64[ns]', freq=None)
```

Regular date sequences: `pd.date_range`

To create a sequence of dates, Pandas offers the `pd.date_range` method. It works similarly to Python's built-in `range` and `np.range`, taking in a start point, an end point, and a step size:

In []:

```
pd.date_range('2015-07-03', '2015-07-10')
```

Alternatively, a `date_range` can be created with a start point and the number of periods:

In [57]:

```
pd.date_range('2015-07-03', periods=8)
```

Out[57]:

```
DatetimeIndex(['2015-07-03', '2015-07-04', '2015-07-05', '2015-07-06',  
              '2015-07-07', '2015-07-08', '2015-07-09', '2015-07-10'],  
              dtype='datetime64[ns]', freq='D')
```

The default frequency is daily ('D'). This can be modified by altering the `freq` argument. For instance, to get hourly timestamps:

In [59]:

```
pd.date_range('2015-07-03', periods=8, freq='h')
```

Out[59]:

```
DatetimeIndex(['2015-07-03 00:00:00', '2015-07-03 01:00:00',  
              '2015-07-03 02:00:00', '2015-07-03 03:00:00',  
              '2015-07-03 04:00:00', '2015-07-03 05:00:00',  
              '2015-07-03 06:00:00', '2015-07-03 07:00:00'],  
              dtype='datetime64[ns]', freq='h')
```

Frequencies and offsets

Here is a non-comprehensive list of date frequencies:

- D - Calendar day
- W - Weekly
- ME - Month-end
- YE - Year end
- H - Hourly
- T - Minutes
- S - Seconds
- L - Milliseconds

Resampling

It is often useful to aggregate data over a small period of time when dealing with long time series. This process is called *resampling*. The method is called `.resample`:

In [60]:

```
import pandas as pd

# Create a time series
rng = pd.date_range('2025-01-01', periods=100, freq='D')
ts = pd.Series(range(len(rng)), index=rng)
```

```
# Print time series before resampling  
print(ts)
```

```
2025-01-01    0  
2025-01-02    1  
2025-01-03    2  
2025-01-04    3  
2025-01-05    4  
..  
2025-04-06   95  
2025-04-07   96  
2025-04-08   97  
2025-04-09   98  
2025-04-10   99  
Freq: D, Length: 100, dtype: int64
```

In [61]:

```
# Resample to monthly frequency, taking the mean  
monthly_mean = ts.resample('ME').mean()  
print(monthly_mean)
```

```
2025-01-31    15.0  
2025-02-28    44.5  
2025-03-31    74.0  
2025-04-30    94.5  
Freq: ME, dtype: float64
```

In [62]:

```
# Resample to weekly mean  
weekly_mean = ts.resample('W').mean()  
print(weekly_mean)
```

2025-01-05	2.0
2025-01-12	8.0
2025-01-19	15.0
2025-01-26	22.0
2025-02-02	29.0
2025-02-09	36.0
2025-02-16	43.0
2025-02-23	50.0
2025-03-02	57.0
2025-03-09	64.0
2025-03-16	71.0
2025-03-23	78.0
2025-03-30	85.0
2025-04-06	92.0
2025-04-13	97.5

Freq: W-SUN, dtype: float64